Mercado Aprende: empieza desde cero

Guía para Erik y su hermano. No necesitas saber programar. Edición ampliada: 10 de octubre de 2026.

Imagina una tienda de juguete. Una persona publica una bicicleta y otra marca que la compró. No se cobra dinero ni se organiza un envío. Vamos a aprender con esa tienda.

REGLA: haz un paso, mira el resultado y continúa. No escribas todos los comandos juntos. Si algo falla, consulta el apartado 17.

Ruta rápida: apartados 1 a 5. Ruta con instalación en Windows: apartados 7 a 9. Entender y construir el proyecto: apartados 10 a 14. Compartir cambios: apartados 15 y 16.

Proyecto: https://github.com/erickvera610/mercado-aprende. Es privado: tu hermano necesita aceptar una invitación para abrirlo.

Qué ya se hizo: aplicación Python, páginas HTML, diseño CSS, pruebas automáticas, repositorio privado y configuración para ejecutarlo en Codespaces. La versión original pasó 10 pruebas. Se creó una aplicación nueva inspirada en el concepto de Team Alpha; no se copió su código.

1. Las cinco palabras que necesitas

GitHub: una página web donde guardamos el proyecto y colaboramos. Se abre en el navegador; no tienes que instalar la página.

Git: el sistema que recuerda los cambios. Imagina fotos de cada versión de tu trabajo.

GitHub Desktop: un programa con botones para usar Git. En Windows, su instalación incluye lo necesario para su propio uso de Git. No necesitas instalar Git por separado para seguir la ruta de Desktop.

Python: el programa que entiende las instrucciones de app.py. Flask: una herramienta de Python que ayuda a responder al navegador.

Repositorio: la carpeta del proyecto con su historial. Clonar: llevar una copia con historial a tu computador. ZIP: llevar los archivos sin ese historial.

Codespace: un computador de trabajo en la nube. Terminal: una ventana donde escribes órdenes. Enter: ejecutar una orden. Commit: guardar una versión. Push: subirla. Pull: traer cambios nuevos.

Cuenta GitHub y cuenta Mercado Aprende son distintas. La primera permite acceder al proyecto privado; la segunda sirve para vender y comprar dentro de la aplicación.

2. Crear una cuenta de GitHub

1. Abre Chrome o Edge. En la barra de arriba escribe https://github.com y pulsa Enter.

2. Pulsa Sign up (Registrarse). Si ya tienes cuenta, usa Sign in (Iniciar sesión) y salta al paso 6.

3. Escribe tu correo personal. Crea una contraseña y elige un nombre de usuario. Anota el nombre: Erik lo necesita para invitarte. No compartas tu contraseña.

4. Completa la verificación que muestre GitHub. Abre tu correo y confirma la cuenta usando el mensaje recibido.

5. Elige la cuenta gratuita si GitHub pregunta por un plan. Para este aprendizaje no necesitas contratar un plan.

6. Comprueba que ves tu foto o círculo de perfil arriba a la derecha. Eso indica que ya entraste.

7. Envía a Erik tu nombre de usuario exacto o el enlace de tu perfil, por ejemplo https://github.com/TU-USUARIO. TU-USUARIO es un ejemplo: reemplázalo por el real.

Los nombres o la posición de algunos botones pueden cambiar. Busca el mismo texto o su traducción.

3. Darle acceso al hermano

Estos pasos los hace Erik con su cuenta. No hay que convertir el proyecto en público.

1. Abre https://github.com/erickvera610/mercado-aprende.

2. Pulsa Settings (Configuración) dentro del repositorio. Busca Collaborators en la sección Access. GitHub puede pedirte volver a confirmar tu identidad.

3. Pulsa Add people. Escribe el usuario exacto de tu hermano. Comprueba el perfil antes de seleccionarlo.

4. Pulsa Add o Send invitation según la pantalla. Esta invitación da capacidad de colaborar y modificar código en un repositorio personal; por eso practicaremos en una rama.

5. Tu hermano abre el correo de GitHub o las notificaciones de su cuenta y acepta la invitación.

6. Tu hermano abre el enlace del repositorio con SU cuenta. Debe ver app.py, templates y README.md. Si ve 404, revisa que aceptó y que está en la cuenta correcta.

No le envíes tu contraseña. Invitarlo al código no le entrega tu Codespace ni tus usuarios o productos guardados. Él puede crear su propio entorno.

4. Ejecutar sin instalar nada

Estos pasos los hace tu hermano después de aceptar la invitación. Esta es la ruta más corta.

1. Abre el repositorio. Pulsa el botón verde Code. Abre la pestaña Codespaces.

2. Antes de crear el entorno, revisa en tu cuenta Settings > Billing and licensing la cuota de Codespaces. Usa la cuota gratuita. Si pide pago o la cuota se agotó, detente y usa la ruta local de el apartado 7.

3. Pulsa Create codespace on main. Si ya aparece uno tuyo, ábrelo. Espera hasta que veas archivos a la izquierda y una terminal abajo.

4. La configuración intenta instalar las dependencias y arrancar la página. Si aparece una pregunta de confianza, acepta solo si el repositorio mostrado es este proyecto.

5. Busca Ports (Puertos), junto a Terminal. Busca el puerto 5000 y abre su enlace o el icono de navegador. Mantén la visibilidad Private.

6. Si no aparece el puerto, abre Terminal > New Terminal y escribe esta orden. Pulsa Enter:

```
bash .devcontainer/start.sh
```

7. Vuelve a Ports y abre el 5000. Debes ver mercado. y el catálogo. Al principio está vacío. El enlace del Codespace de Erik no es el de tu propio entorno.

5. Probar la tienda paso a paso

1. Dentro de Mercado Aprende pulsa Crear cuenta. Usa un nombre como vendedor_demo y una contraseña de prueba de al menos 8 caracteres. Luego inicia sesión.

2. Pulsa Vender un producto. Escribe título: Bicicleta azul. Categoría: Deportes. Estado: Usado. Precio: 25.00. Descripción: Bicicleta de prueba para aprender.

3. Envía el formulario. Vuelve al catálogo. Debe aparecer la bicicleta. Prueba buscar Bicicleta y filtrar Deportes.

4. Cierra sesión. Crea otra cuenta, comprador_demo, e inicia sesión con ella. No puedes comprar tu propio producto.

5. Abre la bicicleta. Pulsa el botón de compra simulada. No debes ingresar tarjeta ni enviar dinero.

6. Abre Mi cuenta. La bicicleta debe aparecer entre tus compras y desaparecer del catálogo disponible.

7. Si abres otra copia de la aplicación, estará vacía: cada computador o Codespace tiene su propia base de datos.

Lo aprendido: registrar usuario, guardar un producto, buscar, comprar y consultar el historial.

6. Cerrar y volver mañana

1. Guarda los archivos que hayas editado. Guardar un archivo no equivale a subirlo a GitHub; para eso consulta los apartados 15 y 16.

2. Abre https://github.com/codespaces. Busca TU Codespace. Pulsa los tres puntos y Stop codespace.

3. No basta con cerrar la pestaña del navegador: detén el entorno para evitar consumir tiempo de ejecución innecesario. Los entornos detenidos pueden seguir consumiendo almacenamiento.

4. Para volver, abre la misma página y selecciona tu Codespace. Si la web no responde, ejecuta bash .devcontainer/start.sh y abre Ports > 5000.

5. Usa Stop para pausar. Delete borra el entorno y sus archivos locales, incluidos productos y usuarios que no estén respaldados. No borra el repositorio de GitHub.

Codespaces no es una tienda publicada permanentemente. La dirección funciona mientras el servidor y el entorno estén activos y tengas acceso al puerto privado.

Si prefieres trabajar sin cuota de nube, instala el proyecto en tu computador como se explica a continuación.

7. Instalar en Windows desde cero

Esta ruta usa Windows 10/11. Cierra cualquier instalador que pida un pago inesperado. Las descargas indicadas son oficiales.

1. Visita https://desktop.github.com. Descarga GitHub Desktop para Windows. Abre el instalador descargado y espera a que termine.

2. Abre GitHub Desktop. Pulsa Sign in to GitHub.com. El navegador te permitirá entrar con TU cuenta y autorizar Desktop. Vuelve al programa.

3. Visita https://www.python.org/downloads/windows/. Busca Python 3.12 y un instalador Windows de 64 bits si tu equipo es de 64 bits. El proyecto está preparado para Python 3.12.

4. Ejecuta el instalador de Python. Si aparece Add python.exe to PATH, marca esa casilla. Pulsa Install Now. Si la pantalla es distinta, sigue las instrucciones oficiales para instalar Python 3.12 y el lanzador py.

5. Cierra y abre PowerShell desde el menú Inicio. Escribe lo siguiente y pulsa Enter:

```
py -3.12 --version
```

6. Debe aparecer Python 3.12 seguido de otro número. Si no lo reconoce, revisa la instalación antes de continuar.

Opcional para editar con comodidad: instala Visual Studio Code desde https://code.visualstudio.com. Python ejecuta el proyecto; VS Code solo ayuda a leer y editar.

8. Copiar el proyecto al computador

1. En GitHub Desktop pulsa File > Clone repository. Selecciona URL.

2. En Repository URL pega https://github.com/erickvera610/mercado-aprende. En Local path elige una carpeta fácil, por ejemplo una dentro de Documentos. Pulsa Clone.

3. Si no tienes permiso, acepta primero la invitación y comprueba que Desktop usa la cuenta de tu hermano.

4. Pulsa Repository > Show in Explorer. Debes ver app.py y requirements.txt. Esta es la carpeta del proyecto.

5. Haz clic en la barra de dirección del Explorador. Escribe powershell y pulsa Enter. Se abrirá una terminal dentro de esa carpeta.

6. Comprueba dónde estás:

```
dir
```

Debe aparecer app.py. Si no aparece, no ejecutes el siguiente paso: abre la carpeta correcta.

Alternativa ZIP: en GitHub pulsa Code > Download ZIP. En Windows, clic derecho al ZIP > Extraer todo. Abre la carpeta extraída que contiene app.py. El ZIP permite ejecutar, pero no conserva Git ni su historial y no incluye los datos de tu Codespace.

9. Encender la web en Windows

Escribe una línea, pulsa Enter y espera. No copies las explicaciones dentro de PowerShell.

1. Crea una caja de herramientas privada para este proyecto:

```
py -3.12 -m venv .venv
```

2. Instala las herramientas que el proyecto necesita:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

3. Enciende la aplicación:

```
.\.venv\Scripts\python.exe app.py
```

4. Deja abierta esa ventana. En Chrome escribe http://127.0.0.1:5000. Debes ver la tienda. Sigue el ejercicio de el apartado 5.

5. Para apagarla, vuelve a PowerShell y pulsa Ctrl+C. Para volver mañana, abre PowerShell en la carpeta y repite solo el paso 3.

No necesitas activar el entorno con Activate.ps1: usamos directamente su Python. 127.0.0.1 significa este computador; enviar esa dirección a tu hermano no abre tu página en su equipo.

10. Qué hay en cada carpeta

app.py: el encargado de la tienda. Recibe lo que pide el navegador, revisa los datos y decide qué mostrar.

requirements.txt: lista de herramientas. Flask crea la web; pytest comprueba que funciona.

templates/: las páginas. base.html tiene la navegación común; index.html muestra el catálogo; auth.html registra e inicia sesión; sell.html publica; detail.html muestra un producto; account.html muestra tu cuenta.

static/style.css: colores, tamaños y diseño para computador y móvil. Cambiar aquí un color cambia la apariencia.

instance/market.db: archivo SQLite donde se guardan usuarios y productos al ejecutar. Se crea automáticamente y no se sube a GitHub.

tests/test_market.py: ejercicios que comprueban el código automáticamente con una base temporal. No usan tus productos reales.

.github/workflows/tests.yml: indica a GitHub Actions que ejecute las pruebas al subir cambios.

.devcontainer/: prepara Codespaces y arranca Flask. .gitignore: impide subir base local, entorno y archivos que no deben compartirse.

docs/: este tutorial en PDF y su versión de texto. Los nombres que comienzan con punto también son parte de la configuración del proyecto.

11. Cómo se construyó: la lógica

1. Se decidió el objetivo: publicar y comprar de forma simulada. Luego se separaron cuentas, productos y compras.

2. En app.py se importaron herramientas. import significa traer una herramienta. def significa crear una función, una pequeña receta con nombre.

3. Se escribió create_app(). Esa receta construye Flask, configura la sesión y crea las tablas si aún no existen.

4. Se creó la tabla users con id, username y password. password contiene un hash, no la contraseña original. Un hash permite comprobar la contraseña sin guardarla tal cual.

5. Se creó listings con vendedor, título, categoría, estado, precio en centavos, descripción y comprador. buyer vacío significa disponible.

6. Se añadieron rutas: / muestra catálogo, /register crea cuenta, /login entra, /sell publica, /items/ID muestra un producto, /account muestra tu historial.

7. Flask recibe GET para mostrar formularios y POST para enviar cambios. Antes de un POST se revisa el token CSRF: una señal asociada a tu sesión que protege los formularios.

8. Se protegieron publicación y compra para que solo funcionen con sesión iniciada. El código usa parámetros SQL, escritos con ?, para tratar la entrada del usuario como datos.

12. Cómo se construyó: páginas y pruebas

1. Se escribieron los HTML de templates. render_template une los datos de Python con el HTML que verá el navegador.

2. En base.html se añadió la navegación compartida. Así no hay que repetirla en cada página.

3. En style.css se definieron colores, espacios y reglas para pantallas pequeñas. Python decide; HTML organiza; CSS viste.

4. La venta revisa título, descripción, categoría, estado y precio. Decimal evita errores de redondeo. 25.00 se guarda como 2500 centavos.

5. La compra usa un UPDATE condicional: solo cambia buyer si el producto sigue disponible y no es del comprador. Así no se vende dos veces.

6. Se escribieron pruebas de registro, login, compra, datos inválidos, protección de formularios y permisos. La versión original aprobó 10 pruebas.

7. Se crearon README y tutorial para explicar uso y estructura. Se añadió .gitignore para excluir datos locales y secretos.

8. Se añadió el workflow de GitHub Actions. Al subir código, GitHub prepara Python, instala las dependencias y ejecuta pytest. Una marca verde significa que ese conjunto de pruebas pasó; no garantiza que cualquier uso comercial sea seguro.

13. Repetir la creación desde cero

Esta actividad reconstruye una copia del proyecto completo. Para inventar tu propia tienda línea por línea, lee app.py con los apartados 11 y 12 al lado.

1. Crea una carpeta vacía llamada mi-mercado. Dentro copia app.py y requirements.txt desde el proyecto recibido.

2. Copia completas las carpetas templates y static. Necesitas esos archivos para ver la interfaz; solo app.py no produce toda la tienda.

3. Copia tests, docs, .github, .devcontainer, README.md y .gitignore. El ZIP preparado incluye estos archivos. No copies .venv ni instance.

4. En esa carpeta abre PowerShell. Ejecuta los tres comandos de el apartado 9. Comprueba que la tienda abre.

5. Detén el servidor con Ctrl+C. Ejecuta las pruebas:

```
.\.venv\Scripts\python.exe -m pytest -q
```

6. Debe aparecer 10 passed para el código entregado. Si modificaste algo, usa los mensajes de fallo para encontrar qué cambió.

7. El apartado siguiente explica cómo crear un repositorio nuevo con esa carpeta. Si solo quieres colaborar con Erik, no crees otro: usa el repositorio ya clonado.

14. Cómo se subió y cómo repetirlo

Lo que se hizo en esta entrega: se creó en GitHub el repositorio privado erickvera610/mercado-aprende con un README inicial. Después se subieron los archivos por la conexión de GitHub, creando blobs (archivos), un árbol (lista de archivos) y un commit (versión). Se avanzó main a esa versión, se verificó Actions y se arrancó Codespaces.

Para repetirlo sin esa conexión, utiliza botones de GitHub Desktop:

1. En Desktop pulsa File > Add local repository. Elige mi-mercado. Si indica que no es un repositorio Git, usa la opción create a repository here.

2. Confirma que la carpeta elegida contiene app.py y .gitignore, sin crear por accidente otra carpeta vacía dentro de ella.

3. Revisa Changes. Deben aparecer el código y las plantillas. No deben aparecer .venv, market.db ni contraseñas. Si aparecen, revisa .gitignore antes de guardar.

4. Escribe Primer proyecto Mercado Aprende en Summary. Pulsa Commit to main si hay cambios pendientes.

5. Pulsa Publish repository. Escribe un nombre como mi-mercado-aprende y mantén Keep this code private activado. Publica.

6. Pulsa View on GitHub. Comprueba que ves los archivos. Abre Actions y espera a que las pruebas terminen. Ahora puedes crear un Codespace como en el apartado 4.

15. Cacharrear sin tocar main

Una rama es una mesa de trabajo aparte. Practicaremos cambiar el título de la web en una rama llamada practica-hermano.

1. En Desktop, con mercado-aprende seleccionado, pulsa Fetch origin. Si aparece Pull origin, púlsalo para traer los cambios. Hazlo antes de empezar y con tus cambios previos guardados.

2. Pulsa Current branch > New branch. Escribe practica-hermano. Parte de main y pulsa Create branch.

3. Abre templates/base.html con VS Code o un editor de texto. Busca el fragmento mercado. que se muestra como marca. Reemplaza solo ese fragmento por Mi mercado. Guarda con Ctrl+S.

4. Ejecuta la aplicación. Recarga el navegador. Debes ver el nombre nuevo. Si no cambia, confirma que editaste la copia que estás ejecutando.

5. En Desktop revisa Changes. Debe aparecer el cambio de base.html. Escribe Cambio el nombre de la tienda en Summary y pulsa Commit to practica-hermano.

6. Pulsa Publish branch la primera vez o Push origin después. Esto sube tu rama al repositorio privado.

Un commit guarda una versión local. Push la lleva a GitHub. No necesitas borrar archivos para aprender. No pongas contraseñas o información personal dentro del código.

16. Compartir y revisar el cambio

1. Después de subir la rama, pulsa Create Pull Request en Desktop o abre GitHub y busca Compare & pull request.

2. Revisa que base sea main y compare sea practica-hermano. Escribe un título claro, por ejemplo Cambio el nombre de la tienda.

3. En la descripción escribe qué cambiaste y cómo lo probaste. Pulsa Create pull request. Una Pull Request es una propuesta de cambio.

4. Erik abre la propuesta y revisa Files changed. En Checks comprueba las pruebas. Si está de acuerdo, pulsa Merge pull request y confirma.

5. En Desktop vuelve a main y usa Fetch origin > Pull origin. Tu copia debe mostrar el cambio aprobado. Guarda primero cualquier trabajo pendiente.

En Codespaces también puedes crear una rama en la terminal:

```
git switch -c practica-hermano
```

Después de editar, abre Source Control en la barra izquierda, revisa el archivo, pulsa + para prepararlo, escribe el mensaje y usa Commit y Publish Branch o Sync Changes. Para solicitar revisión, abre GitHub y crea la Pull Request.

No publiques directamente en main para este ejercicio: la rama y la revisión te permiten entender lo que cambió antes de unirlo.

17. Si algo no sale

GitHub muestra 404: inicia sesión con la cuenta invitada y acepta la invitación. El repositorio es privado.

PowerShell no reconoce py: Python 3.12 o su lanzador no está instalado correctamente. Revisa el apartado 7 y abre una terminal nueva.

No encuentra requirements.txt o app.py: estás en la carpeta equivocada. Usa dir y busca app.py. En un ZIP puede haber una carpeta interior más.

ModuleNotFoundError: Flask no se instaló en el Python que usas. Repite la instalación de el apartado 9 y ejecuta con el Python de .venv.

La dirección local no abre: la terminal debe seguir abierta y mostrar el servidor. Revisa el puerto que aparece. Ctrl+C lo apaga.

Port 5000 is in use: probablemente ya hay un servidor. Prueba abrir la web existente. Si es tu servidor anterior, detenlo antes de iniciar otro.

Codespaces no abre la web: inicia el entorno, ejecuta bash .devcontainer/start.sh y abre Ports > 5000. Si falla, mira instance/server.log. Revisa la cuota antes de crear otro entorno.

Formulario expirado: recarga la página e intenta otra vez. Producto no disponible: ya se compró o pertenece a tu propia cuenta.

No aparecen los productos de Erik: cada entorno guarda una base diferente. Reiniciar conserva datos; borrar el Codespace o market.db los elimina.

18. ZIP, acceso y siguiente paso

Recomendación: invitación para aprender juntos. Permite clonar, ver el historial, crear ramas y proponer cambios. ZIP como copia de respaldo o para practicar sin Git.

Para enviar el ZIP: descarga el archivo que acompaña esta guía o usa Code > Download ZIP. Tu hermano debe extraerlo antes de ejecutar. Ese archivo no incluye usuarios, productos ni el historial Git.

Antes de compartir una copia manual, excluye .venv, instance, .env y contraseñas. No envíes tus credenciales de GitHub. Cada persona usa su propia cuenta.

Esta aplicación es educativa. No tiene pagos reales, envíos, fotos de productos o recuperación de contraseñas. Para una tienda comercial se necesita otro trabajo de despliegue y revisión.

Tu primer logro: abre la tienda, crea dos cuentas de prueba, publica un producto, simula la compra y propón un cambio de texto. No necesitas entender todos los archivos para empezar.

Fuentes oficiales consultadas: https://docs.github.com (Inviting collaborators to a personal repository; Creating a codespace for a repository; Cloning a repository from GitHub to GitHub Desktop). Descargas oficiales: https://desktop.github.com y https://www.python.org/downloads/windows/.

Para recibir una invitación falta el usuario exacto de GitHub del hermano. No se ha enviado una invitación sin identificar primero su cuenta.